

CSEN 296B-2 | Week 4.2 Lab Submission

Complete UML Design Package for AstraNotes

Student Name: Atishay Jain | **Date:** April 27, 2026 | **Project:** AstraNotes | **Technical Path:** Python

Overview

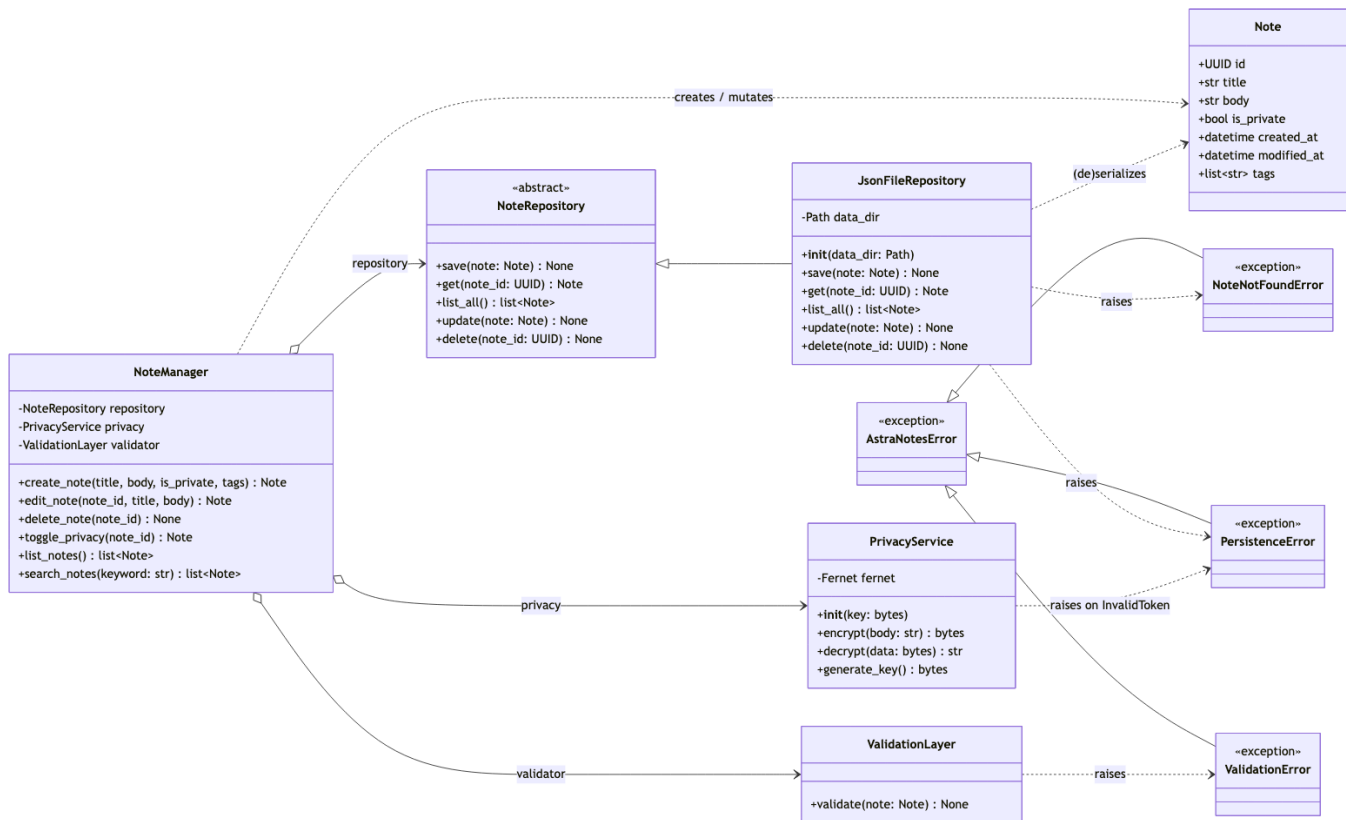
This package contains five UML views of AstraNotes (class, object, use case, activity, deployment), a written rationale tying them together with explicit references to the FR / NFR / SPR IDs from the refined requirement baseline, and an AI reflection covering what the model generated, what I refined, and what I rejected. Every diagram reflects the implemented architecture in the astranotes/ package: Note, NoteRepository (ABC) with JsonFileRepository, PrivacyService, ValidationLayer, the planned NoteManager orchestrator, and the AstraNotesError family of domain exceptions.

Diagram Inventory

#	View	Source file
1	Class diagram	docs/uml/class-diagram.mmd
2	Object diagram (runtime snapshot)	docs/uml/object-diagram.mmd
3	Use case diagram	docs/uml/use-case-diagram.mmd
4	Activity diagram - Create a private note	docs/uml/activity-diagram.mmd
5	Deployment diagram	docs/uml/deployment-diagram.mmd

1. Class Diagram

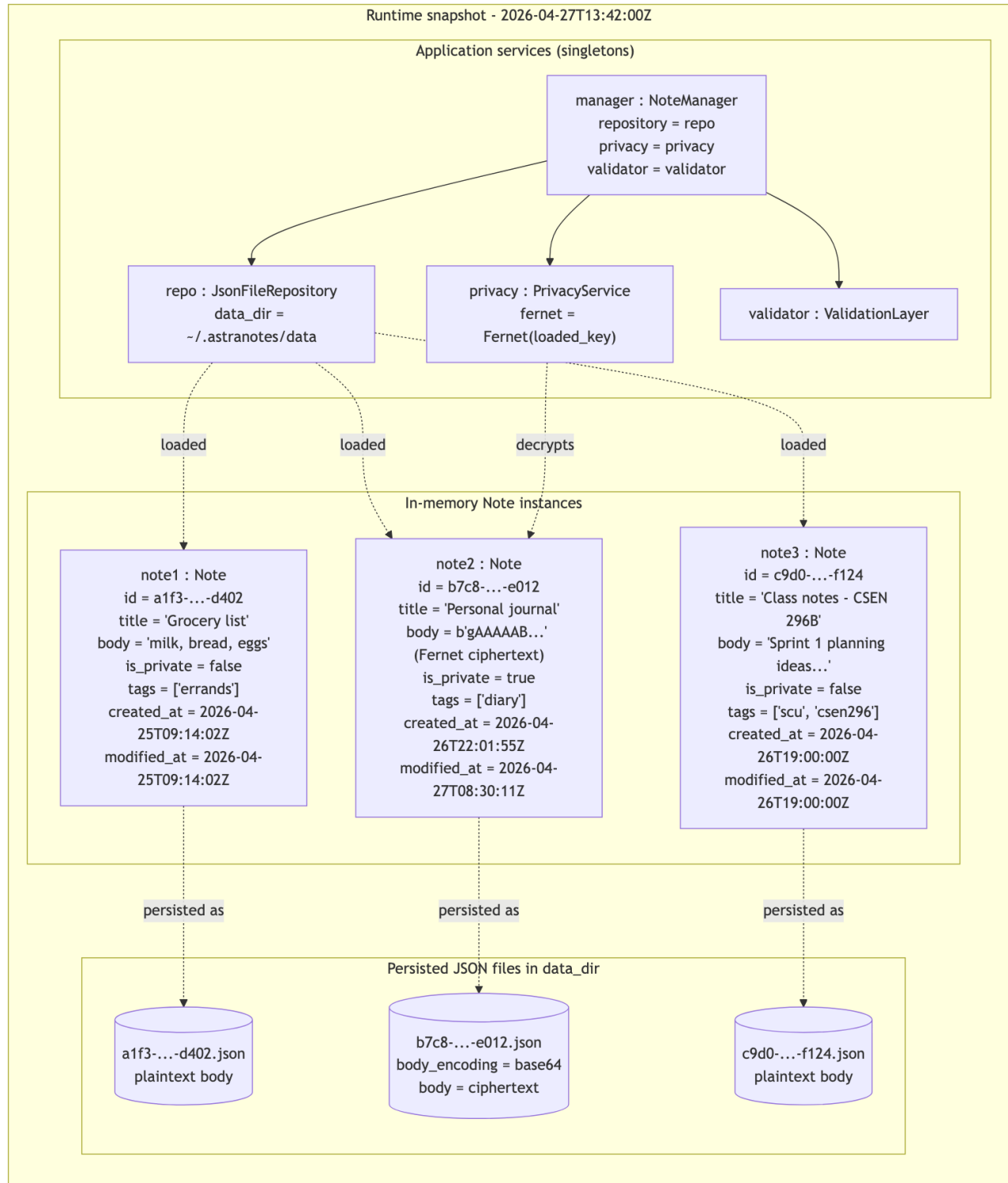
Structural anchor of the package. Shows every implemented class (Note, NoteRepository ABC, JsonFileRepository, PrivacyService, ValidationLayer) plus the planned NoteManager orchestrator, with method signatures, attributes, the AstraNotesError exception hierarchy, and the inheritance / aggregation / dependency edges every other view refers back to.



Rendered from docs/uml/class-diagram.mmd via mermaid-cli. Source listing in Appendix A.

2. Object Diagram

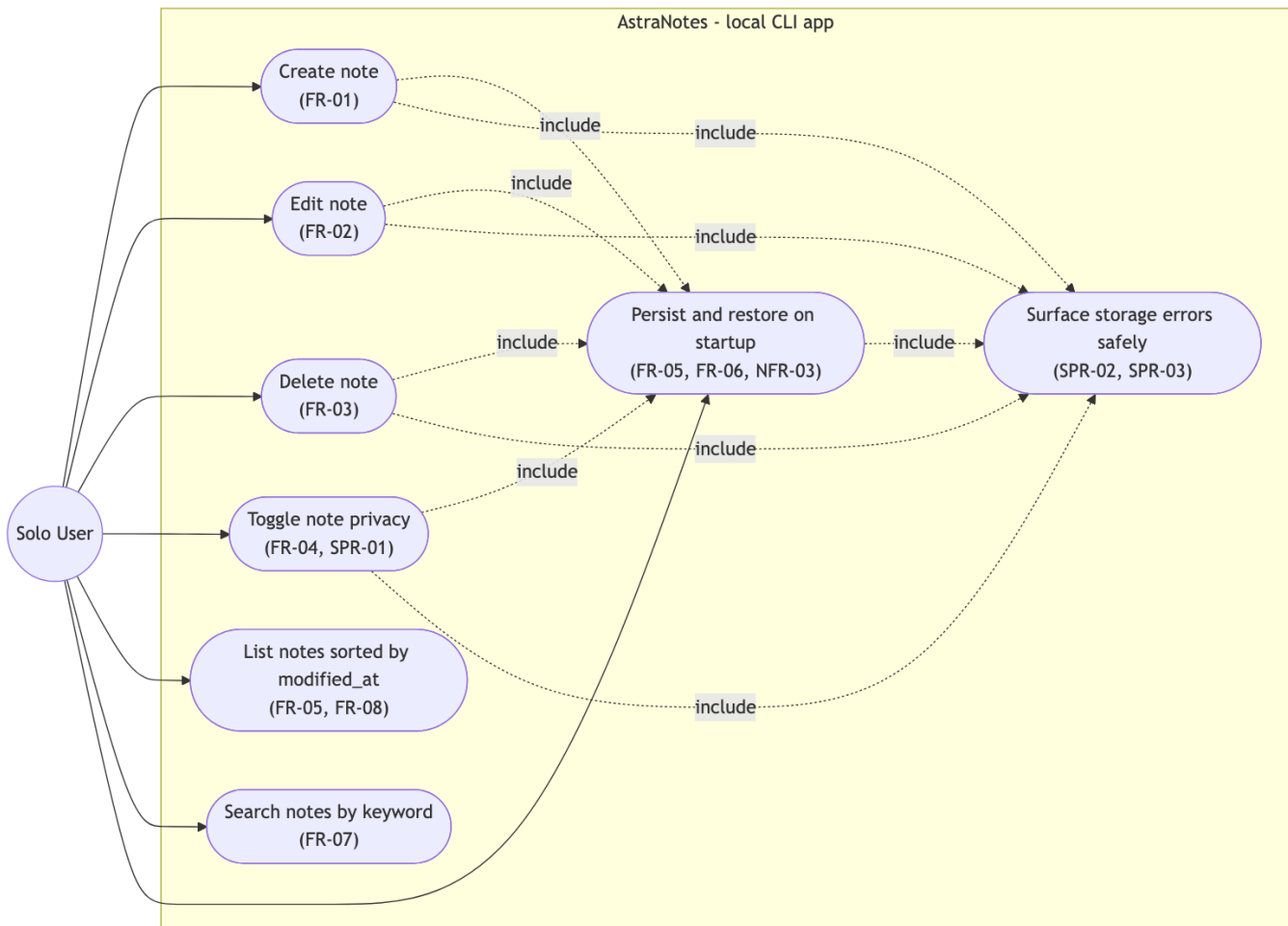
Concrete runtime snapshot at 2026-04-27T13:42:00Z. One instance of each service (manager, repo, privacy, validator) and three Note instances: a public grocery list, a private journal whose body is shown as Fernet ciphertext (SPR-01 in action), and a tagged class note. The Disk subgraph shows the corresponding {uuid}.json files in data_dir, making FR-05 / FR-06 visible in concrete form.



Rendered from docs/uml/object-diagram.mmd via mermaid-cli. Source listing in Appendix A.

3. Use Case Diagram

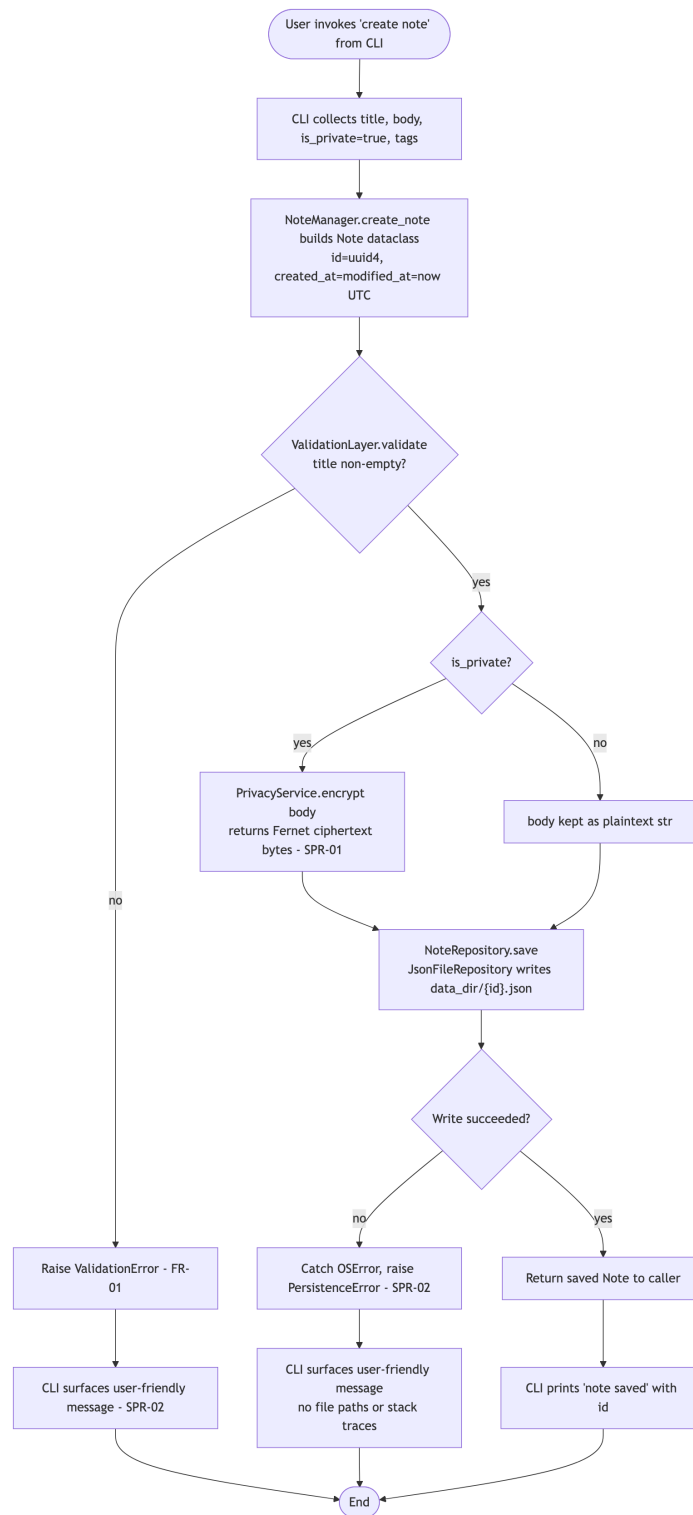
Eight user-visible behaviors inside the AstraNotes system boundary, each tagged with the requirement IDs that justify it. The single actor (Solo User) reflects the explicitly single-user scope. Two cross-cutting use cases (Persist and restore on startup, Surface storage errors safely) are linked from every write-side use case via include arrows, expressing SPR-02 at the use-case level. Mermaid lacks a native UML use-case shape, so stadium nodes inside a labeled subgraph approximate the standard ovals-inside-a-boundary notation.



Rendered from docs/uml/use-case-diagram.mmd via mermaid-cli. Source listing in Appendix A.

4. Activity Diagram - Create a Private Note

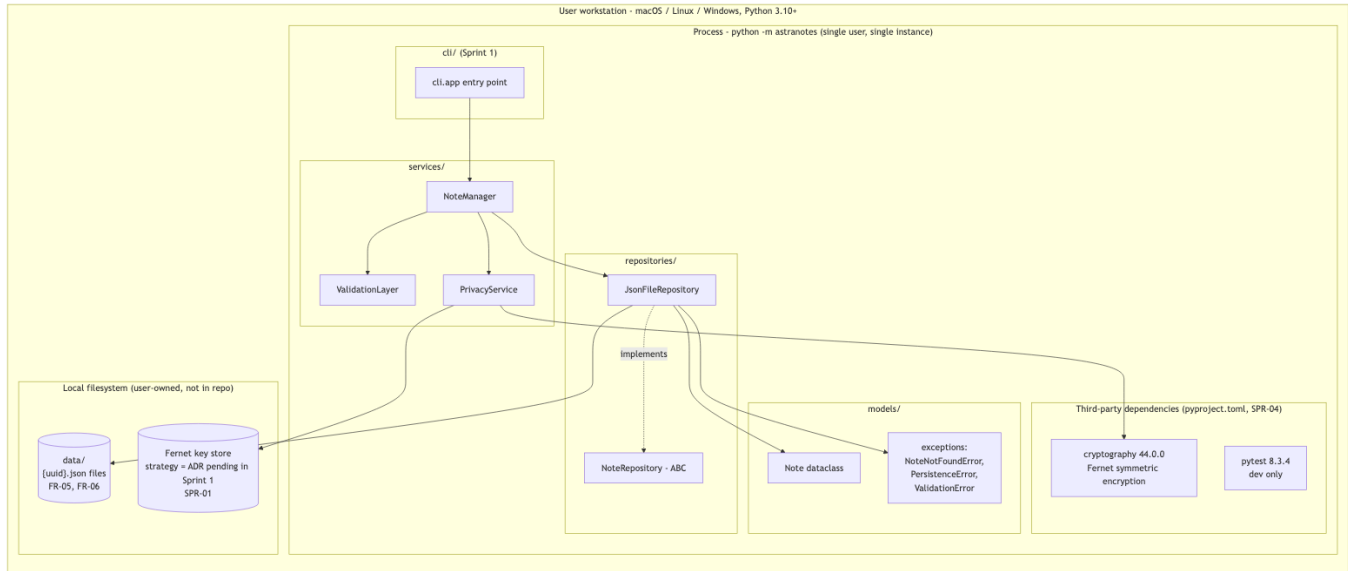
The workflow that exercises every layer in one pass: CLI input, `NoteManager.create_note`, `ValidationLayer.validate` (FR-01 gate), `PrivacyService.encrypt` when `is_private` is true (SPR-01 gate), `JsonFileRepository.save`, and the `OSError-to-PersistenceError` translation step (SPR-02 gate). Every method name and exception class on this diagram is declared on the class diagram with the same signature.



Rendered from docs/uml/activity-diagram.mmd via mermaid-cli. Source listing in Appendix A.

5. Deployment Diagram

Runtime topology: one user workstation (macOS / Linux / Windows, Python 3.10+), one OS process (python -m astranotes), the astranotes/ package decomposed into the same cli/, services/, repositories/, models/ folders that exist in the repo, the third-party cryptography 44.0.0 library (with license / version annotation per SPR-04), and the local filesystem holding the data/ directory and the Fernet key store. The key store carries an explicit ADR pending in Sprint 1 label so the open SPR-01 risk stays visible.



Rendered from docs/uml/deployment-diagram.mmd via mermaid-cli. Source listing in Appendix A.

Design Rationale

AstraNotes - UML Design Package Rationale

Student: Atishay Jain | **Course:** CSEN 296B-2 | **Date:** April 27, 2026 | **Project:** AstraNotes

This document explains how the five UML views in docs/uml/ describe one coherent system. The package is built around the layered architecture from CLAUDE.md: Note (entity), NoteRepository (abstract storage interface) and JsonFileRepository (concrete adapter), PrivacyService (Fernet encryption), ValidationLayer (input rules), NoteManager (orchestrator), and the AstraNotesError family (NoteNotFoundError, PersistenceError, ValidationError).

How the five views fit together

- **Class diagram** (class-diagram.mmd) is the structural anchor. It names every class, attribute, method signature, and relationship used in the rest of the package. NoteRepository is shown as <<abstract>> and JsonFileRepository as its only realization, matching the dependency-injection rule from NFR-02 and the storage choice in FR-05. The exception hierarchy (AstraNotesError parent with three subclasses) appears here so that the activity diagram and the rationale can refer to them by exact name. NoteManager is included even though it is planned for Sprint 1; including it now keeps the orchestration story consistent across all later views.
- **Object diagram** (object-diagram.mmd) is a concrete snapshot at a single instant (2026-04-27T13:42:00Z). It instantiates exactly the classes from the class diagram - one NoteManager, one JsonFileRepository, one PrivacyService, one ValidationLayer, plus three Note instances (a public grocery list, a private journal whose body is shown as Fernet ciphertext, and a tagged class note). The disk subgraph shows the three {uuid}.json files those instances persist to, which makes FR-05 and FR-06 visible in concrete form. The private note's body field shows ciphertext to enforce SPR-01: private bodies never appear in plaintext on disk.
- **Use case diagram** (use-case-diagram.mmd) shows the eight user-visible behaviors, each tagged with the requirement IDs that justify it (FR-01 through FR-08, plus SPR-01, SPR-02, SPR-03, NFR-03). There is exactly one actor - the Solo User - because the project is explicitly single-user (CLAUDE.md "Realistic scope" rule). Two cross-cutting behaviors ("Persist and restore on startup" and "Surface storage errors safely") are linked by <<include>> arrows from every write-side use case, which is the use-case-level expression of the SPR-02 error-translation rule.
- **Activity diagram** (activity-diagram.mmd) drills into one use case - "Create a private note" - because it is the workflow that exercises every layer in one pass. The activity nodes call out the exact methods from the class diagram (NoteManager.create_note, ValidationLayer.validate, PrivacyService.encrypt, NoteRepository.save). Two decision diamonds reflect real branches the code already implements: the validation gate (FR-01), and the OSError-to-PersistenceError translation (SPR-02). The "is_private?" branch shows where SPR-01 takes effect: the body becomes Fernet ciphertext bytes before JsonFileRepository.save writes anything to disk.
- **Deployment diagram** (deployment-diagram.mmd) shows the runtime topology: one user workstation, one OS process (python -m astranotes), the astranotes/ package decomposed into the same cli/, services/, repositories/, models/ folders that exist in the repo, the third-party cryptography library (with a license/version comment per SPR-04), and the local filesystem holding the data/ directory and the Fernet key store. The key store node carries an explicit "ADR pending in Sprint 1" annotation to keep the SPR-01 risk visible until the Architecture Decision Log is updated.

Consistency claims (cross-checked)

- Every class on the class diagram appears either as an instance on the object diagram or as a deployable component on the deployment diagram.
- Every method called in the activity diagram (`create_note`, `validate`, `encrypt`, `save`) is declared on the class diagram with matching signature.
- The exception names used in the activity diagram (`ValidationError`, `PersistenceError`) are declared in the class diagram's exception hierarchy.
- The use case "Toggle note privacy (FR-04, SPR-01)" maps to `NoteManager.toggle_privacy` on the class diagram; the use case "Search notes by keyword (FR-07)" maps to `NoteManager.search_notes`.
- The directory subgraphs in the deployment diagram (`models/`, `services/`, `repositories/`, `cli/`) match the actual project layout in `CLAUDE.md` and the `astranotes/` folder on disk.
- The single-actor scope on the use case diagram matches the single-process, single-workstation topology on the deployment diagram.

Requirements coverage map

Requirement	View(s) where it is visible
FR-01 (create / non-empty title)	Class (<code>ValidationLayer.validate</code>), Use case UC1, Activity (validation gate)
FR-02 (edit)	Class (<code>NoteManager.edit_note</code>), Use case UC2
FR-03 (delete)	Class (<code>NoteRepository.delete</code> , <code>NoteManager.delete_note</code>), Use case UC3
FR-04 (toggle privacy)	Class (<code>NoteManager.toggle_privacy</code>), Use case UC4, Activity (is_private branch)
FR-05, FR-06 (one file per note, metadata)	Object (Disk subgraph), Deployment (data/ node), Class (<code>JsonFileRepository</code>)
FR-07 (keyword search)	Class (<code>NoteManager.search_notes</code>), Use case UC6
FR-08 (sort by modified_at)	Class (<code>NoteManager.list_notes</code>), Use case UC5
NFR-02 (separation of concerns)	Class (ABC + concrete), Deployment (each layer in its own folder)
NFR-03 (startup under 3s)	Use case UC7, Deployment (single local process, no network)
SPR-01 (encrypt private bodies)	Class (<code>PrivacyService</code>), Activity (Encrypt step), Object (note2 ciphertext), Deployment (Fernet key store)
SPR-02 (translate storage errors)	Class (exception hierarchy), Use case UC8, Activity (Surface message steps)
SPR-03 (testable units)	Class (each component has a single, mockable interface)
SPR-04 (licensed third-party deps)	Deployment (cryptography 44.0.0, pytest 8.3.4 with license note)

NFR-01 (500-note benchmark) is not visible in any single diagram - it is a runtime-performance assertion better demonstrated by a benchmark harness in Sprint 1 than by a UML view, and is noted here so reviewers know it is intentionally out of scope.

Design tradeoffs worth flagging

- The use case diagram is rendered with Mermaid `flowchart` because Mermaid does not yet have a native UML use-case shape. Stadium-shaped nodes inside a labeled subgraph are the closest faithful approximation, and the `<<include>>` semantics are preserved with dotted "include" edges.

- NoteManager is in every diagram even though no production code for it exists yet. This is deliberate: the package's value is showing the target architecture, and adding it later would force reissuing all five diagrams. The class diagram's method list defines the contract Sprint 1 must implement.
- The Fernet key store appears as a separate node on the deployment diagram with an "ADR pending" label rather than being elided. The unresolved key-management strategy is the single largest known risk from Sprint Zero, and burying it would be misleading.

AI Reflection

What AI helped generate. The first-pass scaffold for each Mermaid file (class diagram syntax, flowchart layout for the use case and activity views, the deployment subgraph nesting). Picking flowchart with stadium nodes as the closest faithful approximation for a UML use-case view (Mermaid does not support that shape natively) was an AI suggestion I accepted. The requirements coverage matrix in the rationale was sketched by AI from the requirement IDs in CLAUDE.md.

What I refined. I rewrote the class diagram's exception block to use the actual `AstraNotesError` parent class from `astranotes/models/exceptions.py` rather than the flat list AI initially produced. I added the `InvalidToken` to `PersistenceError` translation edge for `PrivacyService` because it is in the code but the first draft missed it. I expanded the activity diagram's failure branches to include the SPR-02 user-friendly message step on both the `ValidationError` and the `PersistenceError` paths, since the original draft only handled the happy path. The object diagram's Disk subgraph (showing the actual `{uuid}.json` files) was added by me to make FR-05 and FR-06 visible in a concrete instance, not just declarable in the schema.

What I rejected. AI suggested adding an `Administrator` actor and an "Export to cloud" use case to the use case diagram. Both rejected as scope creep - the project is explicitly single-user and local-only per CLAUDE.md. AI suggested modeling each note's tags as a separate `Tag` class with a many-to-many relationship to `Note`. Rejected as over-modeling: the implemented `Note` dataclass stores tags as `list[str]`, and inventing a `Tag` class in the UML would diverge from real code. AI suggested removing `NoteManager` from the class diagram because it is not yet implemented. Rejected: leaving it out would force a complete reissue once Sprint 1 lands the orchestrator, and the package's job is to describe the target architecture, not just today's code.

Appendix A - Mermaid Source

Each diagram above was rendered from the corresponding .mmd file below using @mermaid-js/mermaid-cli. The source is included so a reviewer can re-render or edit any view without losing fidelity.

docs/uml/class-diagram.mmd

```
%% AstraNotes - Class Diagram
%% Reflects implemented architecture from CLAUDE.md.
%% NoteManager is planned for Sprint 1 (orchestrator). All other classes exist in code today.
```

```
classDiagram
    direction LR

    class Note {
        +UUID id
        +str title
        +str body
        +bool is_private
        +datetime created_at
        +datetime modified_at
        +list~str~ tags
    }

    class NoteRepository {
        <<abstract>>
        +save(note: Note) None
        +get(note_id: UUID) Note
        +list_all() list~Note~
        +update(note: Note) None
        +delete(note_id: UUID) None
    }

    class JsonFileRepository {
        -Path data_dir
        +__init__(data_dir: Path)
        +save(note: Note) None
        +get(note_id: UUID) Note
        +list_all() list~Note~
        +update(note: Note) None
        +delete(note_id: UUID) None
    }

    class PrivacyService {
        -Fernet fernet
        +__init__(key: bytes)
        +encrypt(body: str) bytes
        +decrypt(data: bytes) str
        +generate_key() bytes
    }

    class ValidationLayer {
        +validate(note: Note) None
    }

    class NoteManager {
        -NoteRepository repository
        -PrivacyService privacy
        -ValidationLayer validator
        +create_note(title, body, is_private, tags) Note
        +edit_note(note_id, title, body) Note
        +delete_note(note_id) None
        +toggle_privacy(note_id) Note
        +list_notes() list~Note~
        +search_notes(keyword: str) list~Note~
    }

    class AstraNotesError {
```

```

    <<exception>>
}
class NoteNotFoundError {
    <<exception>>
}
class PersistenceError {
    <<exception>>
}
class ValidationError {
    <<exception>>
}

NoteRepository <|-- JsonFileRepository
AstraNotesError <|-- NoteNotFoundError
AstraNotesError <|-- PersistenceError
AstraNotesError <|-- ValidationError

NoteManager o--> NoteRepository : repository
NoteManager o--> PrivacyService : privacy
NoteManager o--> ValidationLayer : validator
NoteManager ..> Note : creates / mutates
JsonFileRepository ..> Note : (de)serializes
JsonFileRepository ..> NoteNotFoundError : raises
JsonFileRepository ..> PersistenceError : raises
PrivacyService ..> PersistenceError : raises on InvalidToken
ValidationLayer ..> ValidationError : raises

```

docs/uml/object-diagram.mmd

```

%% AstraNotes - Object Diagram
%% Concrete runtime snapshot at 2026-04-27T13:42:00Z showing live instances of the
%% classes from class-diagram.mmd. Three Note instances: one public, one private
%% (body shown as Fernet ciphertext), one tagged class note.

```

flowchart TB

```

    subgraph Snapshot["Runtime snapshot - 2026-04-27T13:42:00Z"]
        direction TB

        subgraph Services["Application services (singletons)"]
            manager["manager : NoteManager<br/>repository = repo<br/>privacy = privacy<br/>validator = validator"]
            repo["repo : JsonFileRepository<br/>data_dir = ~/.astranotes/data"]
            privacy["privacy : PrivacyService<br/>fernet = Fernet(loaded_key)"]
            validator["validator : ValidationLayer"]
        end

        subgraph Notes["In-memory Note instances"]
            n1["note1 : Note<br/>id = a1f3-...-d402<br/>title = 'Grocery list'<br/>body = 'milk, bread, eggs'<br/>"]
            n2["note2 : Note<br/>id = b7c8-...-e012<br/>title = 'Personal journal'<br/>body = b'gAAAAAB...' (Ferne"]
            n3["note3 : Note<br/>id = c9d0-...-f124<br/>title = 'Class notes - CSEN 296B'<br/>body = 'Sprint 1 pla"]
        end

        subgraph Disk["Persisted JSON files in data_dir"]
            f1["f1('a1f3-...-d402.json<br/>plaintext body')"]
            f2["f2('b7c8-...-e012.json<br/>body_encoding = base64<br/>body = ciphertext')"]
            f3["f3('c9d0-...-f124.json<br/>plaintext body')"]
        end
    end

    manager --> repo
    manager --> privacy
    manager --> validator

    repo -->|loaded| n1
    repo -->|loaded| n2
    repo -->|loaded| n3

    privacy -->|decrypts| n2

    n1 -->|persisted as| f1
    n2 -->|persisted as| f2
    n3 -->|persisted as| f3

```

docs/uml/use-case-diagram.mmd

```
%% AstraNotes - Use Case Diagram
%% Mermaid does not support native UML use-case shapes; flowchart with stadium
%% nodes mimics the standard "ovals inside a system boundary" notation.
%% Single actor (Solo User) per single-user scope (CLAUDE.md).

flowchart LR
    user(("Solo User"))

    subgraph System["AstraNotes - local CLI app"]
        UC1(["Create note<br/>(FR-01)"])
        UC2(["Edit note<br/>(FR-02)"])
        UC3(["Delete note<br/>(FR-03)"])
        UC4(["Toggle note privacy<br/>(FR-04, SPR-01)"])
        UC5(["List notes sorted by modified_at<br/>(FR-05, FR-08)"])
        UC6(["Search notes by keyword<br/>(FR-07)"])
        UC7(["Persist and restore on startup<br/>(FR-05, FR-06, NFR-03)"])
        UC8(["Surface storage errors safely<br/>(SPR-02, SPR-03)"])
    end

    user --> UC1
    user --> UC2
    user --> UC3
    user --> UC4
    user --> UC5
    user --> UC6
    user --> UC7

    %% include relationships - every write goes through persistence and error handling
    UC1 -. include .-> UC7
    UC2 -. include .-> UC7
    UC3 -. include .-> UC7
    UC4 -. include .-> UC7
    UC1 -. include .-> UC8
    UC2 -. include .-> UC8
    UC3 -. include .-> UC8
    UC4 -. include .-> UC8
    UC7 -. include .-> UC8
```

docs/uml/activity-diagram.mmd

```
%% AstraNotes - Activity Diagram
%% Workflow: "Create a private note" - exercises ValidationLayer, PrivacyService,
%% JsonFileRepository, and the SPR-02 error-translation gate. Method names match
%% class-diagram.mmd exactly.

flowchart TD
    Start([User invokes 'create note' from CLI]) --> Collect[CLI collects title, body, is_private=true, tags]
    Collect --> Build[NoteManager.create_note builds Note dataclass<br/>id=uuid4, created_at=modified_at=now UTC]
    Build --> Validate{ValidationLayer.validate<br/>title non-empty?}

    Validate -- no --> RaiseV[Raise ValidationError - FR-01]
    RaiseV --> Surface1[CLI surfaces user-friendly message - SPR-02]
    Surface1 --> End1([End])

    Validate -- yes --> PrivacyCheck{is_private?}
    PrivacyCheck -- yes --> Encrypt[PrivacyService.encrypt body<br/>returns Fernet ciphertext bytes - SPR-01]
    PrivacyCheck -- no --> KeepPlain[body kept as plaintext str]

    Encrypt --> Save
    KeepPlain --> Save["NoteRepository.save<br/>JsonFileRepository writes data_dir/{id}.json"]

    Save --> WriteOk{Write succeeded?}
    WriteOk -- no --> RaiseP[Catch OSError, raise PersistenceError - SPR-02]
    RaiseP --> Surface2[CLI surfaces user-friendly message<br/>no file paths or stack traces]
    Surface2 --> End2([End])

    WriteOk -- yes --> Return[Return saved Note to caller]
    Return --> Confirm[CLI prints 'note saved' with id]
```

Confirm --> End

docs/uml/deployment-diagram.mmd

```
%% AstraNotes - Deployment Diagram
%% Local-first, single-user Python CLI app. One node (user workstation), one
%% process (python -m astranotes), one external dependency (cryptography), and
%% the local filesystem holding JSON note files plus the Fernet key (key store
%% strategy is pending Sprint 1 ADR per Sprint Zero plan).

flowchart TB
    subgraph Workstation["User workstation - macOS / Linux / Windows, Python 3.10+"]
        direction TB

        subgraph Process["Process - python -m astranotes (single user, single instance)"]
            direction TB

            subgraph CLI_Layer["cli/ (Sprint 1)"]
                CLI[cli.app entry point]
            end

            subgraph Services["services/"]
                MGR[NoteManager]
                VAL[ValidationLayer]
                PRV[PrivacyService]
            end

            subgraph Repos["repositories/"]
                BASE[NoteRepository - ABC]
                JFR[JsonFileRepository]
            end

            subgraph Models["models/"]
                NOTE[Note dataclass]
                EXC[exceptions:<br/>NoteNotFoundError,<br/>PersistenceError,<br/>ValidationError]
            end

            subgraph Deps["Third-party dependencies (pyproject.toml, SPR-04)"]
                CRYPT[cryptography 44.0.0<br/>Fernet symmetric encryption]
                PYTEST[pytest 8.3.4<br/>dev only]
            end
        end

        subgraph FS["Local filesystem (user-owned, not in repo)"]
            DATA[("data/<br/>{uuid}.json files<br/>FR-05, FR-06")]
            KEY[("Fernet key store<br/>strategy = ADR pending in Sprint 1<br/>SPR-01")]
        end
    end

    CLI --> MGR
    MGR --> VAL
    MGR --> PRV
    MGR --> JFR
    JFR -.implements.-> BASE
    JFR --> NOTE
    JFR --> EXC
    PRV --> CRYPT
    PRV --> KEY
    JFR --> DATA
```